

FINANCEGUARD

End-to-End DevSecOps Capstone Project Plan

CI/CD + Security + Infrastructure as Code + Monitoring

Finance Industry Reference Implementation on AWS and Kubernetes

Final Project for the Running Training Batch

Version 1.0 | 9 June 2026

Mandatory Data Handling Rule

The implementation must use synthetic transactions and tokenized identifiers only. Real payment card data, bank account numbers, customer PII, production credentials, and regulated data are prohibited in the training environment.

Reference baseline: the provided Terraform + Ansible project plan, expanded into a finance-grade DevSecOps capstone.

Document Control

Field	Value
Document purpose	Define the required architecture, implementation sequence, security controls, evidence, documentation, and evaluation criteria for the training batch final project.
Target audience	DevOps, Cloud, Platform Engineering, SRE, Application Security, and DevSecOps trainees.
Reference implementation	AWS, Amazon EKS, GitHub Actions, Argo CD, Terraform, Helm, Kubernetes, and an open-source observability/security stack.
Delivery model	Recommended three-repository GitOps model in one GitHub organization. A controlled monorepo exception is permitted only when documented.
Recommended duration	Eight one-week implementation sprints plus final demonstration and evidence review.
Data classification	Training / synthetic data. No real financial, cardholder, customer, or production data.
Compliance posture	Control-aligned educational implementation; it is not a certification, audit opinion, or declaration of regulatory compliance.
Required outcome	A reproducible, auditable, secure, observable, and recoverable delivery platform with working code and complete documentation.

How to Use This Plan

- Mandatory requirements use the words **MUST**, **REQUIRED**, or **BLOCKING** and are part of the minimum passing scope.
- Recommended requirements use **SHOULD** and represent the expected finance-grade implementation when budget and lab capacity permit.
- Optional items are stretch goals and may be used to differentiate advanced teams.
- Each phase includes required repository artifacts, implementation steps, acceptance criteria, and audit evidence. Teams should complete phases in sequence.
- The reference stack is opinionated to keep grading consistent. Alternative tools are allowed only through a documented Architecture Decision Record (ADR) demonstrating equivalent controls and maintainability.

Document Navigation

Section	Topic
1	Executive Summary
2	Project Goal, Scenario, and Objectives
3	Scope, Assumptions, and Success Measures
4	Selected Technology Stack and Alternatives
5	Target Architecture and Trust Boundaries
6	GitHub Repository and Governance Model
7	End-to-End Delivery Process
8	Implementation Phases and Step-by-Step Guidance
9	Security Architecture and Quality Gates
10	Monitoring, Logging, Tracing, and SRE
11	Resilience, Backup, Disaster Recovery, and Rollback
12	Financial-Industry Standards and Control Mapping
13	Testing and Validation Strategy
14	Documentation, Evidence, and Runbooks
15	Delivery Schedule, Roles, and Governance
16	Grading Rubric and Definition of Done
17	Final Demonstration and Submission Checklist
18	Risks, Constraints, and Stretch Goals
A-G	Appendices: repository trees, settings, workflow templates, commands, evidence pack, and references

1. Executive Summary

FinanceGuard is a training-grade digital payment transaction platform used to demonstrate a complete DevSecOps operating model. The application accepts synthetic payment requests, applies basic validation and risk rules, creates an idempotent ledger entry, emits audit events, and exposes health and transaction-status endpoints. The technical goal is not to build a real payment processor; it is to prove that infrastructure, software, security, deployment, monitoring, and operational evidence can be managed as code.

The reference solution provisions an AWS environment with Terraform, deploys containerized services to Amazon EKS, builds and scans artifacts with GitHub Actions, promotes immutable image digests through a GitOps repository, deploys with Argo CD and Helm, and collects metrics, logs, and traces using OpenTelemetry, Prometheus, Grafana, Loki, Tempo, and CloudWatch. Security is integrated from pull request to runtime through secret scanning, SAST, SCA, IaC scanning, image scanning, SBOM generation, signing and provenance, policy-as-code, Kubernetes admission controls, runtime detection, cloud-native security services, and time-bound exception handling.

The design is aligned to financial-sector expectations such as separation of duties, least privilege, immutable evidence, change approval, traceable deployments, operational resilience, and third-party/supply-chain risk management. The project maps its controls to PCI DSS v4.0.1, NIST CSF 2.0, NIST SSDF 1.1, NIST SP 800-53 Rev. 5 Release 5.2.0, the 2024 FFIEC Development, Acquisition, and Maintenance booklet, EU DORA, ISO/IEC 27001:2022, OWASP ASVS 5.0, OWASP Top 10:2025, CIS Benchmarks, Kubernetes Pod Security Standards, and SLSA 1.2. These mappings guide evidence collection; they do not make the training environment compliant or certified.

Required Reference Stack

GitHub Actions for CI; Argo CD for CD; Terraform for IaC; Helm for application packaging; Amazon EKS and ECR; AWS Secrets Manager and KMS; Semgrep/CodeQL, Gitleaks, Checkov, Trivy, Syft, Cosign, OWASP ZAP, Kyverno, and Falco; Prometheus, Grafana, Alertmanager, Loki, Tempo, OpenTelemetry, and CloudWatch.

2. Project Goal, Scenario, and Objectives

2.1 Project Goal

Build a secure, fully automated, end-to-end DevSecOps solution for a Kubernetes-based finance application. The solution must provision infrastructure, validate and package software, enforce security controls, promote immutable artifacts, deploy through GitOps, observe service health and business transactions, support controlled rollback and recovery, and generate an evidence pack suitable for an internal audit walkthrough.

2.2 Business Scenario

A fictitious financial institution is modernizing a small payment transaction service. The service receives a synthetic request, validates the payload, applies a simple risk rule, records an immutable transaction event, returns a status, and publishes telemetry. The institution requires every infrastructure and application change to be reviewed, tested, scanned, attributable to a person and commit, deployed through an approved workflow, and observable after release.

- Transaction API: receives synthetic payment requests and enforces idempotency using a client-supplied key.
- Risk Rule Service: applies deterministic training rules such as amount thresholds, velocity counters, or country allow/deny lists.
- Ledger Worker: writes append-only transaction events to PostgreSQL and emits a status update. Records are not a real accounting ledger.
- Optional Web UI: displays synthetic transaction status and system health. It must not expose secrets or sensitive identifiers.
- Audit Event Stream: produces structured events with actor, correlation ID, transaction token, action, timestamp, outcome, and deployment version.

2.3 Core Objectives

- Automate cloud infrastructure provisioning and teardown through reviewed Terraform code.
- Implement secure, repeatable Kubernetes platform configuration and application deployment.
- Use GitHub Actions for CI and Argo CD for pull-based GitOps CD.
- Integrate security controls at source, dependency, IaC, container, deployment, cluster, cloud, and runtime layers.
- Build once and promote the same signed container digest across environments.
- Prevent long-lived cloud credentials in GitHub by using OIDC federation and least-privilege IAM roles.
- Provide end-to-end metrics, logs, traces, alerting, dashboards, SLOs, and operational runbooks.
- Demonstrate controlled release strategies, rollback, backup restoration, and resilience testing.
- Produce traceable compliance evidence, ADRs, risk records, threat models, and audit-ready documentation.
- Maintain clear separation of duties between application source, platform infrastructure, and environment configuration.

2.4 Learning Outcomes

Domain	Trainee must demonstrate
CI/CD	Reusable workflows, protected environments, approval gates, immutable artifact promotion, release traceability, and rollback.
IaC	Modular Terraform, remote encrypted state, policy checks, drift detection, repeatable environment creation, and cost-aware teardown.
Security	Threat modeling, secure SDLC gates, software supply-chain controls, Kubernetes and AWS hardening, runtime detection, and evidence management.
Monitoring/SRE	Telemetry instrumentation, dashboards, actionable alerts, SLOs, incident triage, and post-deployment verification.
Finance controls	Segregation of duties, auditability, least privilege, encryption, change control, resilience, and third-party risk awareness.
Documentation	Architecture, ADRs, runbooks, control mapping, risk acceptance, operating procedures, and a complete README-driven setup.

3. Scope, Assumptions, and Success Measures

3.1 In Scope

- Synthetic finance application source code, unit/integration tests, Dockerfiles, API specification, and seed data.
- AWS networking, IAM, EKS, ECR, RDS PostgreSQL, KMS, Secrets Manager, CloudWatch, logging, backup, and security services provisioned with Terraform.
- GitHub repository governance, Actions workflows, OIDC federation, environments, approvals, CODEOWNERS, branch protection, and dependency management.
- Argo CD, Helm charts, environment overlays, policy-as-code, namespace/RBAC design, network policies, pod security, secrets integration, and release promotion.
- SAST, SCA, secret scanning, license checks, IaC scanning, container scanning, SBOMs, signatures, provenance/attestations, DAST, runtime detection, and vulnerability exception handling.
- OpenTelemetry instrumentation, metrics, logs, traces, dashboards, alerts, SLOs, synthetic monitoring, and incident runbooks.
- Backup/restore, deployment rollback, failure injection, evidence collection, risk register, control mapping, and final demonstration.

3.2 Out of Scope

- Processing or storing real cardholder data, bank account data, customer PII, production secrets, or regulated financial records.
- Claiming PCI DSS, ISO 27001, SOC 2, FFIEC, DORA, NYDFS, or any other certification or regulatory compliance.
- Building real settlement, clearing, reconciliation, anti-money-laundering, KYC, or fraud-decisioning capabilities.
- Production-grade multi-region active-active architecture unless selected as a stretch goal.
- Purchasing commercial tools. Enterprise products may be described as alternatives but are not mandatory for the lab.

3.3 Deployment Profiles

Profile	Minimum topology	Use case	Required caveat
Training minimum	One AWS account; one EKS cluster; isolated dev and staging namespaces; separate Terraform state files; production simulated through a protected namespace.	Cost-constrained batch delivery.	Document that namespace isolation is not equivalent to separate production accounts/clusters.
Standard project	Separate non-production and production AWS accounts; separate EKS clusters; centralized logging/security account or shared-services account.	Recommended final-project target when lab accounts permit.	Define cross-account roles, log aggregation, and promotion controls.
Finance reference	AWS Organizations landing zone; security, log archive, shared services, dev, staging, and production accounts; private clusters; centralized SIEM and break-glass access.	Architecture reference and advanced teams.	Higher cost and operational complexity; use only when the team can operate it safely.

3.4 Success Measures and Training Targets

Measure	Target	Evidence
Environment reproducibility	A clean dev environment can be created and destroyed from code without console-only dependencies.	Terraform plan/apply logs, state backend, and teardown workflow.
PR quality gate	All mandatory checks complete before merge; bypasses are disabled or documented as break-glass events.	Branch rules and successful/failed workflow runs.
Artifact integrity	Each release has an immutable image digest, SBOM, vulnerability result, signature, and provenance/attestation.	ECR metadata and release evidence bundle.
Deployment traceability	Running pods can be traced to Git commit, workflow run, image digest, GitOps commit, approver, and deployment timestamp.	Labels/annotations, Argo history, GitHub deployment records.
Security threshold	No unresolved critical or high fixable vulnerabilities in released artifacts; every exception is approved, owned, and time-bound.	Scan reports and exception records.

Measure	Target	Evidence
Availability SLO	Training target: 99.9% monthly availability for the transaction API.	Prometheus recording rules and Grafana dashboard.
Latency SLO	Training target: p95 API latency below 500 ms under the agreed test load.	k6 result and dashboard.
Error rate	Training target: fewer than 1% failed valid requests during performance tests.	RED metrics and k6 report.
Recovery	Training RTO <= 60 minutes and RPO <= 15 minutes for the database restore exercise.	Timed restore report and evidence.
Rollback	A failed release is automatically or manually rolled back within 10 minutes.	Argo history, Git revert, alert, and incident timeline.

4. Selected Technology Stack and Alternatives

4.1 Selection Principles

- Prefer declarative, version-controlled, API-driven tools with strong audit trails and automation interfaces.
- Separate build responsibilities from deployment responsibilities: CI produces trusted artifacts; GitOps CD reconciles desired state.
- Use short-lived identity and workload identity instead of static credentials.
- Use open standards for portability: OCI images, SBOM formats, OIDC, OpenTelemetry, Prometheus, Kubernetes, Helm, and in-toto/SLSA attestations.
- Pin versions and action commit SHAs; upgrade through controlled pull requests and regression tests.
- Choose one primary tool per control objective to reduce noisy duplication; use an alternative only when it closes a documented gap.
- Keep commercial finance-enterprise options visible, but ensure the batch can complete the mandatory path with open-source or cloud-native services.

4.2 Reference Stack Summary

Capability	Selected technologies	Status
Source control / governance	GitHub organization, protected repositories, CODEOWNERS, Issues/Projects, environments	Mandatory
Continuous integration	GitHub Actions reusable workflows; GitHub-hosted runners for untrusted PRs; isolated self-hosted runners only for private networking	Mandatory
Continuous delivery	Argo CD with AppProjects/ApplicationSets; Helm charts; promotion through GitOps pull requests	Mandatory
Infrastructure as Code	Terraform modules; S3 backend with KMS encryption, versioning, and native lockfile; TFLint and Checkov	Mandatory
Configuration automation	Ansible for bootstrap/hardening tasks not cleanly handled by Terraform/Helm; ansible-lint and Molecule where used	Recommended
Cloud / platform	AWS, Amazon EKS, ECR, RDS PostgreSQL, ALB, WAF, KMS, Secrets Manager, CloudWatch	Mandatory
Secrets	GitHub OIDC to AWS; AWS Secrets Manager; External Secrets Operator or Secrets Store CSI Provider; no secrets in Git	Mandatory
Application security	Semgrep CE; CodeQL where licensed; Dependabot/Renovate; Trivy filesystem/SCA; Gitleaks	Mandatory
IaC / manifest security	TFLint, Checkov, Trivy config, kubeconform, kube-linter, Conftest/OPA	Mandatory
Container supply chain	BuildKit/buildx, ECR immutable tags, Syft SBOM, Trivy image scan, Cosign signing, GitHub artifact attestation/SLSA provenance	Mandatory
DAST / API testing	OWASP ZAP baseline/full scan; Schemathesis optional for OpenAPI fuzzing	Mandatory / Optional
Admission / policy	Kyverno policies for pod security, image source, resource limits, and signed-image verification	Mandatory
Runtime security	Falco; GuardDuty EKS protection; Security Hub aggregation; Inspector scanning	Recommended
Observability	OpenTelemetry Collector, Prometheus, Grafana, Alertmanager, Loki, Tempo, CloudWatch	Mandatory
SIEM / enterprise option	Splunk Enterprise Security, Elastic Security, Microsoft Sentinel, or Datadog Security Monitoring	Optional
Performance / resilience	k6, Kubernetes probes, PodDisruptionBudgets, autoscaling, backup restore, optional Chaos Mesh/Litmus	Mandatory / Optional

4.3 CI/CD Tool Decision Matrix

Tool	Role	Why selected / when to use	Important controls
GitHub Actions	CI orchestration and controlled infrastructure workflows	Native to GitHub, reusable workflows, environment approvals, OIDC federation, artifacts, security integrations, and accessible training experience.	Minimal GITHUB_TOKEN permissions; OIDC; approved actions; full-length SHA pinning; concurrency; protected environments; no privileged untrusted runners.
Argo CD	Pull-based Kubernetes CD	Continuously compares live and desired state, exposes drift, supports declarative configuration, RBAC, AppProjects, sync waves, health checks, and rollback history.	Separate GitOps repo; SSO/RBAC; least-privilege projects; signed commits; protected config branches; disable direct production kubectl for normal operations.
GitLab CI/CD	Enterprise alternative	Good integrated registry, security scanning, and environments where GitLab is the organizational standard.	Equivalent OIDC, approvals, protected runners, and GitOps separation required.
Jenkins	Legacy/regulated network alternative	Use only when on-premises integration or existing enterprise operations justify it.	Controller isolation, ephemeral agents, plugin governance, credential management, patching, and audit logging add substantial operational burden.
Azure DevOps Pipelines	Alternative for Microsoft-centric estates	Good enterprise controls and Azure integration.	Service connections should use workload identity federation; equivalent artifact signing and GitOps controls required.

4.4 IaC and Configuration Tool Decision Matrix

Tool	Use	Decision rationale	Alternative / caveat
Terraform	Primary infrastructure provisioning	Mature AWS provider ecosystem, modules, plans, state, policy integrations, and broad enterprise adoption.	OpenTofu is an acceptable alternative with an ADR; Pulumi is allowed only when the batch can support language/runtime governance.
Ansible	Host/bootstrap and procedural configuration	Useful for runner hosts, bastions, OS hardening, one-time bootstrap, and validation tasks; idempotent roles provide repeatability.	Do not use Ansible to replace Kubernetes declarative resources or Terraform-owned cloud resources.
Helm	Kubernetes packaging	Templated, versioned releases with values per environment and broad ecosystem support.	Kustomize may be added for overlays; avoid opaque templates that cannot be validated.
Terraform Cloud/Enterprise	Enterprise remote execution/policy option	Adds private registry, runs, policy, approvals, and governance.	Not required; the lab uses encrypted S3 state and GitHub Actions.
AWS CloudFormation/CDK	AWS-native alternative	Useful where native service support or organizational standards require it.	Must provide equivalent plan review, drift detection, modularity, and security scanning.

4.5 Security Tool Selection Matrix

Control area	Selected	Why	Enterprise alternatives
Secrets in source	Gitleaks	Fast pre-commit and CI scanning; custom rules; blocks verified secrets.	GitHub secret scanning/push protection or TruffleHog.
SAST	Semgrep CE	Accessible, rules-as-code, fast PR feedback.	CodeQL/GHAS where licensed; SonarQube Enterprise for centralized governance.
SCA / dependency risk	Dependabot or Renovate + Trivy fs	Automated updates plus vulnerability and license reporting.	Snyk, Mend, Black Duck, or JFrog Xray.
IaC misconfiguration	Checkov + TFLint	Policy checks plus Terraform correctness/style.	Trivy config, tfsec legacy, Terraform Enterprise policy.
Kubernetes validation	kubeconform + kube-linter	Schema validation plus best-practice checks before deployment.	Polaris or Datree.
Policy-as-code	Conftest in CI + Kyverno in cluster	Same intent before and during admission; Kubernetes-native policies and reports.	OPA Gatekeeper where Rego is the enterprise standard.
Image vulnerability	Trivy	Broad ecosystem support, SBOM integration, filesystem and image scanning.	Grype, Clair, ECR enhanced scanning/Inspector.

Control area	Selected	Why	Enterprise alternatives
SBOM	Syft with CycloneDX/SPDX output	Simple generation for OCI images and filesystems.	Trivy SBOM or native build-system SBOM.
Signing / verification	Cosign	OIDC/keyless or KMS-backed signing, OCI registry support, admission verification.	Notation/Notary v2 where enterprise policy requires it.
Provenance	GitHub artifact attestations and SLSA-compatible provenance	Links artifact to workflow/source and supports verification.	Tekton Chains or in-toto layouts.
DAST	OWASP ZAP	Accessible baseline and authenticated scan options.	Burp Suite Enterprise, Invicti, or StackHawk.
Runtime detection	Falco + AWS detections	Kubernetes runtime rules plus cloud threat detections.	Sysdig Secure, Prisma Cloud, Wiz Runtime, or Aqua.

4.6 Monitoring and Observability Decision Matrix

Technology	Purpose	Reason for selection	Enterprise option
OpenTelemetry	Standard instrumentation and telemetry collection	Vendor-neutral traces, metrics, and logs; prevents hard coupling to one backend.	Commercial OTel backends such as Datadog, Dynatrace, New Relic, Splunk Observability.
Prometheus	Metrics and alert evaluation	Kubernetes-native ecosystem and PromQL; supports SLO and platform metrics.	Amazon Managed Service for Prometheus or enterprise monitoring platform.
Grafana	Dashboards and exploration	Unified visualization for metrics, logs, and traces.	Amazon Managed Grafana or enterprise observability suite.
Alertmanager	Routing, grouping, inhibition, escalation	Keeps alert rules as code and demonstrates on-call routing.	PagerDuty/Opsgenie/ServiceNow Event Management integration.
Loki	Application and Kubernetes logs	Cost-conscious label-based log aggregation suitable for training.	OpenSearch, Elastic, Splunk, CloudWatch Logs Insights.
Tempo	Distributed tracing backend	Integrates with OpenTelemetry and Grafana.	AWS X-Ray or commercial tracing backend.
CloudWatch	AWS control-plane and service telemetry	Native EKS/RDS/ALB/WAF metrics and logs; required for cloud incidents.	Central SIEM or observability platform receives exports.

5. Target Architecture and Trust Boundaries

5.1 Logical Architecture

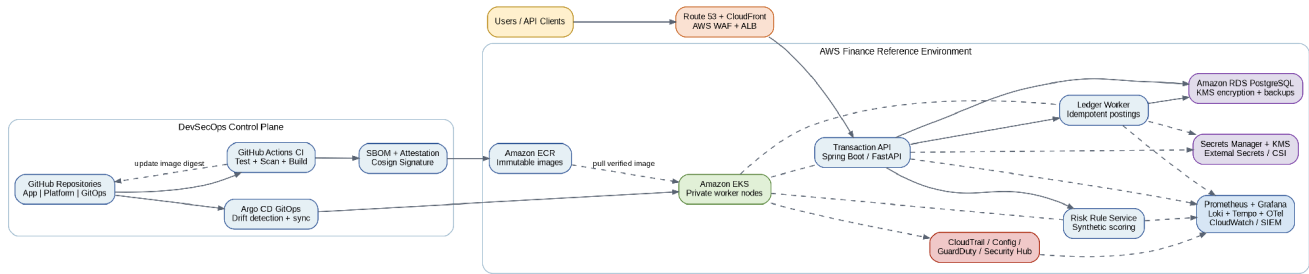


Figure 1. FinanceGuard reference architecture and DevSecOps control plane.

5.2 Architecture Layers

Layer	Components	Responsibilities
Edge and ingress	Route 53, CloudFront optional, AWS WAF, ALB/Ingress	TLS termination, rate limiting, IP controls, managed WAF rules, request logging, and protected entry to the cluster.
Kubernetes platform	Amazon EKS, managed node groups or Karpenter, namespaces, RBAC, network policies, Pod Security Standards, autoscaling	Workload scheduling, isolation, identity, scaling, availability, and admission enforcement.
Application	Transaction API, Risk Rule Service, Ledger Worker, optional UI	Synthetic transaction processing, idempotency, risk rules, audit events, health endpoints, and telemetry.
Data	RDS PostgreSQL, encrypted backups/snapshots, optional Redis	Durable append-only events and application state; training data only; encrypted in transit and at rest.
Secrets and keys	AWS Secrets Manager, KMS, External Secrets Operator or CSI provider, EKS Pod Identity	Central secret lifecycle, encryption keys, least-privilege workload access, and rotation.
Artifact supply chain	GitHub Actions, ECR, Syft, Trivy, Cosign, attestations	Build, scan, inventory, sign, attest, store, and promote immutable image digests.
GitOps CD	GitOps repository, Argo CD, Helm	Desired-state promotion, drift detection, synchronization, health checks, and controlled rollback.
Observability	OpenTelemetry, Prometheus, Grafana, Loki, Tempo, CloudWatch, Alertmanager	Metrics, logs, traces, dashboards, alerting, SLOs, audit support, and post-deployment verification.
Cloud security	CloudTrail, Config, GuardDuty, Security Hub, Inspector, WAF logs	Control-plane audit, configuration drift, threat detection, vulnerability findings, and centralized security visibility.

5.3 Trust Boundaries

Boundary	Risk statement	Required controls
TB-1 Internet to edge	Untrusted clients reach only the approved HTTPS endpoint.	WAF, TLS, rate limiting, request validation, no public Kubernetes API or nodes.
TB-2 Edge to workloads	Ingress routes only to approved services and ports.	IngressClass controls, security groups, NetworkPolicy, service accounts, namespace isolation.
TB-3 Workloads to data	Only authorized service accounts access RDS and secrets.	Pod Identity/IRSA, Secrets Manager, security groups, TLS, separate DB roles, no shared admin credentials.
TB-4 CI to AWS	CI requests short-lived AWS credentials.	GitHub OIDC trust restricted by organization, repository, branch/environment, audience, and workflow.
TB-5 CI to artifact registry	Only release workflow can push immutable artifacts.	Dedicated ECR role, immutable tags, digest references, scan/sign/attest before promotion.
TB-6 GitOps to cluster	Argo CD reconciles desired state; humans do not deploy directly in normal operations.	Argo project allowlists, SSO/RBAC, protected GitOps branches, audit logs, break-glass procedure.
TB-7 Telemetry to backends	Telemetry must not leak sensitive or secret data.	Structured logging schema, redaction, tokenization, sampling, access controls, retention policy.

Boundary	Risk statement	Required controls
TB-8 Admin access	Privileged access is exceptional and attributable.	MFA, SSO, least privilege, just-in-time access where available, session logging, break-glass review.

5.4 Network and Account Design

- Use at least two Availability Zones and private subnets for EKS nodes and RDS. Public subnets contain only internet-facing load balancers and NAT gateways when required.
- Deny public access to RDS, ECR endpoints, internal services, and worker nodes. Prefer VPC endpoints for ECR, S3, STS, CloudWatch Logs, EKS Auth, Secrets Manager, and KMS where cost permits.
- Restrict the EKS API endpoint to private access or approved CIDRs. All administrator access must use SSO/MFA and auditable roles.
- Use separate security groups for load balancers, EKS nodes, pods where supported, and RDS. Permit only explicit flows.
- Centralize CloudTrail, Config, WAF, ALB, and security findings. In the standard/finance profiles, send logs to a dedicated log archive account with immutable retention controls.
- Use separate Terraform state prefixes/backends and separate IAM roles for bootstrap, plan, non-production apply, and production apply.

5.5 Application Non-Functional Requirements

Requirement	Minimum implementation
Idempotency	The transaction API accepts an idempotency key and returns the original result for retried requests.
Auditability	Each business action writes a structured audit event with timestamp, actor/client, correlation ID, tokenized transaction ID, action, outcome, and application version.
Data minimization	Only synthetic data and tokenized identifiers are stored. Secrets and tokens are never logged.
Encryption	TLS for external traffic; TLS to RDS; KMS encryption for RDS, ECR, S3 state, log archives, and secrets.
Availability	At least two application replicas, readiness/liveness/startup probes, PodDisruptionBudget, anti-affinity/topology spread, and graceful shutdown.
Resource governance	Requests and limits for every container; autoscaling based on CPU and one application metric where feasible.
Safe failure	Circuit-breaking/retry logic must be bounded; retries must not create duplicate ledger entries.
Telemetry	OpenTelemetry trace context propagated across services and included in structured logs.
Versioning	API and application release versions are explicit; database migrations are versioned and backward-compatible for rolling deployments.

6. GitHub Repository and Governance Model

6.1 Recommended Repository Model

The project SHOULD use one GitHub organization with three repositories. This implements separation of concerns and prevents the CI workflow that builds application code from directly controlling production desired state.

Repository	Contents	Primary owners	Deployment permission
financeguard-app	Application services, tests, Dockerfiles, OpenAPI, application Helm chart source, CI workflows.	Application team + AppSec reviewers.	Can push images to ECR and open a PR to the GitOps repository; cannot deploy directly.
financeguard-platform	Terraform modules/stacks, Ansible roles, platform Helm releases, cluster bootstrap, policy library, platform workflows.	Platform/Cloud team + Security.	Can plan all environments; applies require environment-specific roles and approvals.
financeguard-gitops	Environment-specific Helm values, Argo Applications/ApplicationSets, approved image digests, sync waves, promotion history.	Release/Platform team + application approvers.	Argo CD reads this repository; production branch/path requires approval.

6.2 Mandatory Repository Controls

- Protect the default branch. Require pull requests, successful status checks, conversation resolution, and review by CODEOWNERS.
- Require at least one approval for non-production and two distinct approvals for production-impacting changes. One approver must be outside the author team for production.
- Disable direct pushes, force pushes, and branch deletion on protected branches. Restrict bypass permissions.
- Use GitHub Environments for dev, staging, and production. Production requires reviewers, prevents self-approval, and uses environment-scoped OIDC trust.
- Set GITHUB_TOKEN to read-only by default and grant explicit job-level permissions. Grant id-token: write only to jobs that need OIDC.
- Allow only GitHub-owned or approved actions. Pin all third-party actions to full-length commit SHAs and review updates through Dependabot/Renovate.
- Enable Dependabot alerts/updates and GitHub secret scanning/push protection where available. Gitleaks remains mandatory for all repositories.
- Add CODEOWNERS for workflows, Terraform, security policies, GitOps production paths, and database migrations.
- Use signed commits/tags for releases where practical. Release tags must be immutable and semantically versioned.
- Create issue templates for vulnerability exception, production change, incident, and risk acceptance. Use labels and milestones for audit traceability.

6.3 Branching, Release, and Promotion Model

Area	Required approach
Development flow	Short-lived feature branches, pull requests to main, trunk-based development, feature flags for incomplete functionality.
Application release	A merge to main creates a candidate image. A signed semantic tag or approved release event creates the release artifact.
Artifact identity	Use git SHA and semantic version tags, but deploy by immutable image digest. Never rebuild for staging or production.
Environment promotion	Promotion is a pull request changing only the approved digest and environment values in the GitOps repository.
Production approval	GitHub protected environment and GitOps CODEOWNERS approval. Evidence must identify author, reviewers, workflow, digest, and outcome.
Rollback	Revert the GitOps commit or select the previous approved digest; Argo CD reconciles the prior desired state. Database migrations require a forward-fix/expand-contract plan.

7. End-to-End Delivery Process

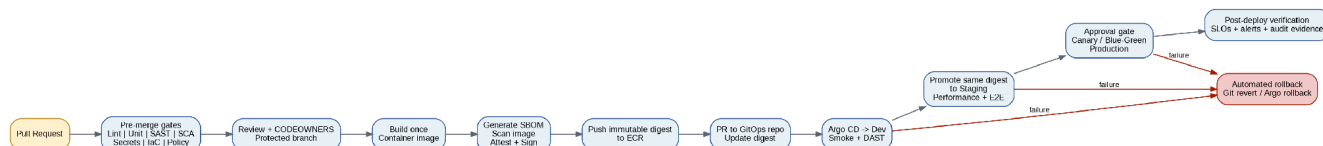


Figure 2. Build once, verify, sign, and promote the same digest through GitOps.

7.1 Required Process Flow

1. A developer creates a short-lived branch, updates code/tests/docs, and runs pre-commit checks locally.
2. A pull request triggers linting, unit tests, secret scanning, SAST, SCA, license checks, IaC/manifest validation, and policy tests. No cloud write credentials are available to untrusted PR jobs.
3. CODEOWNERS review security-sensitive files. Required checks and review rules must pass before merge.
4. The trusted main-branch workflow builds the container in an isolated runner, records dependencies, generates an SBOM, scans the image, and executes integration tests.
5. The workflow creates provenance/attestation and signs the image using keyless OIDC or an AWS KMS-backed key. The immutable digest is pushed to ECR.
6. The workflow opens a pull request to the GitOps repository updating the dev environment to the new image digest. The pull request includes scan summary, SBOM link, test results, and source commit.
7. Argo CD reconciles dev. Health checks, smoke tests, API tests, and ZAP DAST run. Failure triggers rollback or prevents promotion.
8. The same digest is promoted to staging through a GitOps PR. End-to-end, performance, resilience, and security regression tests run.
9. Production promotion requires protected-environment approval and required reviewers. Deployment uses canary, blue-green, or controlled rolling strategy.
10. Post-deployment verification checks error rate, latency, saturation, business-event success, logs, traces, and alerts. A release is not complete until verification passes.
11. Every release produces an evidence bundle: source SHA, workflow run, approvers, image digest, SBOM, signature verification, scan results, GitOps commit, Argo history, tests, and monitoring snapshot.
12. Rollback is performed by reverting desired state to the previous approved digest. The incident/change record captures cause, decision, impact, and corrective action.

7.2 Build-Once/Promote-Many Rule

Blocking Rule

The production image **MUST** be byte-for-byte identical to the image tested in dev and staging. Environment differences belong in configuration and secrets, not in rebuilt images. Deployment manifests **MUST** reference an immutable digest.

8. Implementation Phases and Step-by-Step Guidance

The following phases are the core assignment. Each phase contains required activities, repository artifacts, acceptance criteria, and evidence. Teams may work in parallel, but the control dependencies must be respected.

8.1 Phase 0 - Team Charter, Governance, and Project Initialization

Implementation steps

1. Create the GitHub organization and the three repositories. Add team membership using least privilege.
2. Create a team charter containing members, roles, communication channel, working agreement, escalation path, and definition of ready/done.
3. Create a project board with epics for application, IaC, CI, CD, security, observability, resilience, documentation, and evidence.
4. Add LICENSE, SECURITY.md, CONTRIBUTING.md, CODE_OF_CONDUCT.md, pull-request template, issue templates, and CODEOWNERS.
5. Configure protected branches, required checks placeholders, environment reviewers, action allowlist, and default read-only workflow permissions.
6. Create ADR-001 selecting AWS/EKS/GitHub Actions/Argo CD/Terraform and document considered alternatives.
7. Create a data-handling statement prohibiting real financial data and defining synthetic test-data generation.
8. Create initial risk register and assumptions log.

Required repository artifacts

- README.md
- docs/team-charter.md
- docs/adr/ADR-001-platform-stack.md
- docs/risk-register.md
- .github/CODEOWNERS
- .github/pull_request_template.md
- .github/ISSUE_TEMPLATE/*
- SECURITY.md
- CONTRIBUTING.md

Acceptance criteria

- A new contributor can understand the project, governance, and setup from the repositories without verbal guidance.
- Protected-branch and environment settings are demonstrated during review.
- No member has unnecessary organization-owner or repository-admin access.

8.2 Phase 1 - Application Baseline, Data Model, and Threat Model

Implementation steps

1. Implement the transaction API, risk service, and ledger worker or use the instructor-provided starter application.
2. Use synthetic data. Model transaction tokens, status, amount, currency, created time, risk result, and audit fields. Do not model or store PAN, CVV, real accounts, names, or addresses.
3. Implement idempotency, request validation, bounded timeouts/retries, structured error responses, and health/readiness endpoints.
4. Instrument services with OpenTelemetry and propagate correlation/trace IDs.
5. Create an OpenAPI specification and contract tests. Add unit tests and database integration tests using Testcontainers or an equivalent isolated database.
6. Create a data-flow diagram and STRIDE threat model covering source, CI, artifact registry, GitOps, cluster, data store, secrets, telemetry, and admin access.
7. Define security requirements using OWASP ASVS 5.0 categories appropriate to the application.
8. Create Dockerfiles using multi-stage builds, minimal non-root runtime images, fixed base-image digests, read-only filesystem where possible, and no package manager in the final image.

Required repository artifacts

- services/transaction-api/*

- services/risk-service/*
- services/ledger-worker/*
- openapi/financeguard.yaml
- docs/architecture/data-flow.md
- docs/security/threat-model.md
- docs/security/asvs-requirements.md
- tests/*

Acceptance criteria

- Unit test coverage target is documented and met; critical domain paths have tests.
- API rejects invalid data and duplicate retries do not create duplicate ledger events.
- Containers run as non-root and pass local image/config scans.
- Threat model identifies assets, actors, trust boundaries, threats, controls, owners, and residual risks.

8.3 Phase 2 - Bootstrap, Remote State, and GitHub-to-AWS Identity

Implementation steps

1. Create a one-time bootstrap stack for the Terraform state S3 bucket, KMS key, versioning, access logging, public-access block, and native S3 lockfile.
2. Create separate state keys/prefixes for dev, staging, production, and shared services. Do not share a single state file across environments.
3. Configure GitHub OIDC in AWS. Create separate least-privilege IAM roles for plan, non-production apply, production apply, image publishing, and evidence upload.
4. Restrict role trust by GitHub organization, repository, branch/tag/environment, workflow identity where feasible, and audience. Do not store AWS access keys in GitHub.
5. Implement Terraform formatting, validation, documentation, TFLint, Checkov, and plan workflows. Publish plans as review artifacts without exposing secrets.
6. Require manual approval for apply. Production apply must use a protected GitHub environment and separate role.
7. Add drift-detection workflow that runs on a schedule and opens an issue or sends an alert for unexpected changes.

Required repository artifacts

- bootstrap/*
- terraform/backend.hcl.example
- terraform/modules/*
- .github/workflows/terraform-plan.yml
- .github/workflows/terraform-apply.yml
- .github/workflows/terraform-drift.yml
- docs/runbooks/terraform-state-recovery.md

Acceptance criteria

- Terraform state is encrypted, versioned, locked, and inaccessible publicly.
- OIDC succeeds without long-lived AWS credentials.
- PRs show a reviewed plan and scans; applies are impossible from untrusted PRs.
- A deliberate out-of-band change is detected by the drift workflow.

8.4 Phase 3 - AWS Landing Zone, Network, Data, and EKS Provisioning

Implementation steps

1. Create reusable Terraform modules for VPC, subnets, routing, endpoints, security groups, IAM, EKS, ECR, RDS, KMS, Secrets Manager, logging, and security services.
2. Provision at least two Availability Zones. Place worker nodes and RDS in private subnets. Expose only the approved load balancer.
3. Use EKS managed node groups or Karpenter with hardened AMIs, encrypted volumes, IMDSv2, restricted node IAM, and autoscaling.
4. Enable EKS control-plane logs, envelope encryption for Kubernetes secrets where applicable, access entries/RBAC, audit logging, and restricted API endpoint access.
5. Create ECR repositories with immutable tags, encryption, lifecycle policies, and scan configuration.

6. Create RDS PostgreSQL with KMS encryption, TLS enforcement, backups, deletion protection for the production profile, performance monitoring, and least-privilege database roles.
7. Enable CloudTrail, Config, GuardDuty, Security Hub, Inspector, and appropriate log exports. Configure WAF managed rules for the ingress endpoint.
8. Output only non-sensitive values. Mark sensitive outputs and never echo secrets in CI logs.

Required repository artifacts

- terraform/modules/network
- terraform/modules/eks
- terraform/modules/ecr
- terraform/modules/rds
- terraform/modules/security-baseline
- terraform/environments/{dev,staging,prod}
- docs/architecture/aws-topology.md

Acceptance criteria

- All resources are created by Terraform and tagged with owner, environment, application, cost-center, data-classification, and managed-by.
- RDS and nodes are not publicly reachable.
- EKS control-plane logs and CloudTrail events are visible.
- A clean terraform plan after apply shows no unexpected changes.

8.5 Phase 4 - Kubernetes Platform Bootstrap and Policy Baseline

Implementation steps

1. Bootstrap Argo CD and platform add-ons through a controlled initial process, then hand management to GitOps.
2. Install AWS Load Balancer Controller, ExternalDNS where allowed, cert-manager if required, metrics-server, External Secrets/CSI provider, EBS CSI, and observability components.
3. Create namespaces, ResourceQuotas, LimitRanges, service accounts, RBAC, NetworkPolicies, PodDisruptionBudgets, and Pod Security Admission labels.
4. Install Kyverno and apply policies for non-root, disallow privilege escalation, drop capabilities, read-only root filesystem where compatible, approved registries, required probes, resource requests/limits, and prohibited host access.
5. Install Falco and configure a minimal rule set for shell execution, sensitive file access, privilege escalation, and unusual network/process activity. Route alerts to the selected channel.
6. Create Argo CD AppProjects that restrict source repositories, destinations, namespaces, and cluster-scoped resources.
7. Back up Argo CD configuration and document bootstrap/recovery.

Required repository artifacts

- platform/addons/*
- platform/policies/kyverno/*
- platform/network-policies/*
- platform/rbac/*
- argocd/projects/*
- argocd/applicationsets/*
- docs/runbooks/argocd-recovery.md

Acceptance criteria

- A deliberately non-compliant pod is rejected and a compliant pod is admitted.
- A workload cannot reach an unauthorized namespace or RDS port.
- Argo CD reports drift and can reconcile a changed resource.
- Falco produces a test alert with documented triage steps.

8.6 Phase 5 - Continuous Integration and Secure Pull-Request Gates

Implementation steps

1. Create reusable CI workflows called by each service repository/path. Use concurrency cancellation for superseded PR runs.
2. Run formatting, linting, unit tests, coverage, API contract tests, and Dockerfile linting.

3. Run Gitleaks, Semgrep or CodeQL, dependency/SCA and license checks, Checkov/TFLint, Helm lint, kubeconform, kube-linter, and Conftest policy tests.
4. Separate untrusted PR jobs from trusted build/release jobs. Never expose write tokens, cloud credentials, signing identities, or secrets to forked/untrusted code.
5. Pin third-party actions to full commit SHAs and declare minimal permissions. Use an allowlist and automated update PRs.
6. Upload SARIF or human-readable findings and publish a concise security summary. Fail the workflow according to the gate policy.
7. Cache dependencies without caching secrets or untrusted build outputs across trust boundaries.

Required repository artifacts

- .github/workflows/ci.yml
- .github/workflows/reusable-ci.yml
- .github/dependabot.yml
- .semgrep.yml
- .gitleaks.toml
- .checkov.yml
- policy/ci/*

Acceptance criteria

- A PR containing a test secret is blocked.
- A high-severity IaC misconfiguration is blocked.
- A vulnerable dependency or deliberate SAST issue is detected and handled according to policy.
- Workflow permissions and action pins pass review.

8.7 Phase 6 - Trusted Build, SBOM, Signing, and Artifact Publication

Implementation steps

1. Build containers only after trusted main/release events. Use BuildKit/buildx, deterministic inputs, fixed base-image digests, and isolated ephemeral runners.
2. Run integration tests against the built image. Generate an SBOM in CycloneDX JSON and/or SPDX format using Syft.
3. Scan the final image with Trivy and ECR/Inspector. Fail on critical/high fixable vulnerabilities unless an approved exception exists.
4. Attach SBOM and vulnerability results to the release/evidence location. Include package licenses and base-image metadata.
5. Create a GitHub artifact attestation or SLSA-compatible provenance statement linking source, workflow, builder, and artifact digest.
6. Sign the image with Cosign using keyless OIDC or AWS KMS. Verify the signature before updating GitOps.
7. Push to ECR with semantic version and git SHA tags, enable immutability, and use digest for all deployments.
8. Open the dev GitOps pull request automatically with release notes, evidence links, and digest.

Required repository artifacts

- .github/workflows/release.yml
- scripts/generate-sbom.sh
- scripts/verify-artifact.sh
- docs/security/supply-chain.md
- release-evidence/<version>/*

Acceptance criteria

- A verifier can independently validate the image digest, signature, SBOM, and provenance.
- Re-tagging or overwriting an existing release is prevented.
- The GitOps PR references the exact digest and source commit.

8.8 Phase 7 - GitOps Continuous Delivery and Safe Promotion

Implementation steps

1. Create a Helm chart with values schema, security contexts, probes, resources, PDB, autoscaling, service account, NetworkPolicy, and observability annotations.

2. Create environment overlays/values in the GitOps repository. Store no secrets; reference secret names and external secret resources.
3. Configure Argo CD applications/application sets, automated sync for dev, controlled sync for staging, and approval-gated production promotion.
4. Use sync waves/hooks for namespaces, secrets references, database migrations, services, and verification jobs. Migrations must support rolling deployment.
5. Deploy dev, run smoke/API/DAST tests, and post results to the GitOps PR or deployment record.
6. Promote the same digest to staging and run E2E, performance, and resilience tests.
7. Use canary, blue-green, or controlled rolling production deployment. Configure progressive delivery with Argo Rollouts as an advanced option.
8. Implement automated rollback criteria based on Kubernetes health and key metrics, with a documented manual rollback procedure.

Required repository artifacts

- charts/financeguard/*
- environments/dev/*
- environments/staging/*
- environments/prod/*
- argocd/*
- .github/workflows/promote.yml
- docs/runbooks/deployment-rollback.md

Acceptance criteria

- Direct kubectl deployment is not part of the normal process.
- Argo CD shows desired/live state and deployment history.
- A failed readiness check or error-rate threshold prevents/rolls back promotion.
- The previous digest can be restored through a Git revert.

8.9 Phase 8 - Cloud, Kubernetes, Application, and Runtime Security

Implementation steps

1. Implement least-privilege IAM roles and EKS Pod Identity/IRSA for every AWS-integrated workload. Do not reuse the node role for application access.
2. Mount secrets at runtime from Secrets Manager using External Secrets or CSI. Demonstrate rotation without committing or rebuilding an image.
3. Apply Kubernetes Restricted Pod Security profile or documented compatible exceptions. Enforce NetworkPolicies and egress control.
4. Configure WAF rules, rate limits, TLS policies, security headers, and request-size limits. Add API input validation and authentication/authorization appropriate to the training scope.
5. Use Kyverno to verify image source and, where supported, Cosign signatures/attestations before admission.
6. Route GuardDuty, Security Hub, Inspector, Falco, and WAF findings to a central dashboard/channel. Define severity and escalation.
7. Run an authenticated ZAP scan in staging. Triage findings and create time-bound exceptions only with compensating controls.
8. Conduct a tabletop exercise for compromised GitHub token, compromised image, leaked secret, and suspicious pod execution.

Required repository artifacts

- docs/security/iam-matrix.md
- docs/security/vulnerability-management.md
- docs/security/exception-process.md
- docs/runbooks/secret-rotation.md
- docs/runbooks/security-incident.md
- platform/policies/*

Acceptance criteria

- No application pod receives broad AWS permissions.

- Secret rotation is demonstrated.
- Unsigned/unapproved images or prohibited pod settings are rejected.
- Security findings have owners, severity, response SLA, and evidence.

8.10 Phase 9 - Observability, SLOs, Alerting, and Operational Readiness

Implementation steps

1. Instrument application services with OpenTelemetry SDKs and deploy an OpenTelemetry Collector.
2. Collect Kubernetes/node metrics, application metrics, ALB/RDS/EKS metrics, structured logs, traces, and deployment events.
3. Create dashboards for cluster health, workload health, database, ingress/WAF, CI/CD, security findings, and finance-domain transaction metrics.
4. Define SLIs/SLOs for availability, latency, valid-request success, and ledger completion. Create recording rules and error-budget views.
5. Create actionable alerts with owner, severity, runbook URL, threshold rationale, and inhibition to avoid storms.
6. Implement synthetic monitoring that submits a safe synthetic transaction and verifies end-to-end completion.
7. Redact secrets and sensitive-like fields from logs. Use tokenized IDs and correlation IDs. Define retention for training and production reference profiles.
8. Demonstrate trace-to-log-to-metric navigation for one transaction and one failure.

Required repository artifacts

- observability/prometheus/*
- observability/grafana/dashboards/*
- observability/loki/*
- observability/tempo/*
- observability/otel/*
- docs/sre/slo.md
- docs/runbooks/alerts/*

Acceptance criteria

- Dashboards show RED and USE signals plus business metrics.
- Each paging alert has a working runbook and can be tested.
- A synthetic transaction appears in metrics, logs, and traces with the same correlation ID.
- A deliberate failure produces an alert and an observable recovery.

8.11 Phase 10 - Backup, Restore, Resilience, and Disaster Recovery

Implementation steps

1. Configure RDS automated backups/snapshots and test point-in-time or snapshot restore to an isolated environment.
2. Back up critical Kubernetes/GitOps configuration or prove that the cluster can be reconstructed from Terraform and Git. Store necessary external data safely.
3. Implement PodDisruptionBudgets, topology spread, autoscaling, graceful shutdown, and multi-AZ data configuration appropriate to the profile.
4. Run failure exercises: delete a pod, drain a node, fail a readiness dependency, block an egress path, and deploy a broken version.
5. Measure actual recovery time and data loss against training RTO/RPO targets. Record gaps and corrective actions.
6. Test application and infrastructure rollback. Document database migration rollback/forward-fix constraints.
7. Create a disaster recovery and business continuity runbook including declaration, roles, communications, restore sequence, validation, and return to normal.

Required repository artifacts

- docs/runbooks/database-restore.md
- docs/runbooks/disaster-recovery.md
- docs/resilience/game-day-report.md
- terraform/restore-test/*
- tests/resilience/*

Acceptance criteria

- A restore is completed and validated with timed evidence.
- Loss of one pod/node does not cause unacceptable outage.
- A broken release is rolled back within the target.
- The team documents actual RTO/RPO and improvement actions.

8.12 Phase 11 - Compliance Evidence, Audit Walkthrough, and Handover**Implementation steps**

1. Complete the control mapping in Section 12 and identify evidence for each implemented control.
2. Generate a release evidence pack containing approvals, plans, tests, scans, SBOM, signature verification, provenance, GitOps history, deployment verification, and monitoring snapshots.
3. Create an access review showing GitHub teams, AWS roles, Kubernetes RBAC, Argo CD roles, and database roles.
4. Create a change-management record for the final production-simulated release and an incident record for the failure exercise.
5. Run a peer audit: a team member not responsible for the control follows the documentation and verifies evidence.
6. Remove unused credentials, temporary access, test exceptions, and stale resources. Confirm cost-control teardown instructions.
7. Prepare operational handover: architecture, dependency inventory, upgrade procedure, backup/restore, alert ownership, known risks, and backlog.

Required repository artifacts

- docs/compliance/control-matrix.md
- docs/compliance/evidence-index.md
- docs/access-review.md
- docs/change-records/*
- docs/incidents/*
- docs/handover.md
- evidence/*

Acceptance criteria

- An independent reviewer can trace a release end-to-end.
- Evidence is immutable or integrity-protected and contains no secrets.
- All residual risks and exceptions have owner, rationale, compensating control, and expiry.

8.13 Phase 12 - Final Hardening, Demonstration, and Submission**Implementation steps**

1. Run a complete clean deployment from documentation in a new or reset environment.
2. Resolve all critical/high findings or complete approved exception records. Remove debug endpoints and temporary broad permissions.
3. Run the final release from pull request through production-simulated deployment, verification, alerting, and evidence generation.
4. Execute one rollback and one database restore exercise.
5. Present architecture, tool decisions, threat model, pipeline, GitOps, security findings, dashboards, SLOs, recovery, control mapping, and lessons learned.
6. Submit repository links, tagged release, evidence bundle, final report, demo recording if required, and completed grading self-assessment.

Required repository artifacts

- FINAL-REPORT.md
- DEMO-SCRIPT.md
- SUBMISSION-CHECKLIST.md
- release tag
- evidence/final-release/*

Acceptance criteria

- All mandatory Definition of Done items pass.
- The demo is reproducible and does not depend on undisclosed manual steps.
- No secrets or real data exist in repositories, logs, artifacts, or screenshots.

9. Security Architecture and Quality Gates

9.1 Defense-in-Depth Control Model

Layer	Preventive controls	Detective / response controls
Identity	SSO/MFA, least privilege, OIDC, Pod Identity/IRSA, scoped service accounts, separation of duties.	Access logs, periodic access review, CloudTrail, anomalous IAM findings, break-glass review.
Source	Protected branches, CODEOWNERS, signed releases, secret prevention, dependency pinning.	Secret scanning, SAST, dependency alerts, audit log review.
Build	Trusted event boundary, isolated runners, minimal permissions, pinned actions, deterministic inputs.	Workflow audit, provenance, attestation verification, runner monitoring.
Artifact	Immutable ECR, SBOM, image scan, signature, provenance, approved registry.	Registry scan findings, signature verification, admission reports.
IaC / configuration	Modules, policy-as-code, secure defaults, plan review, protected apply.	Drift detection, Config, Security Hub, Terraform plan review.
Kubernetes	PSS Restricted, Kyverno, RBAC, NetworkPolicy, quotas, secrets integration, non-root containers.	Audit logs, policy reports, Falco, GuardDuty EKS findings, runtime alerts.
Application	ASVS-derived requirements, input validation, idempotency, authn/authz, safe logging, rate limiting.	DAST, API tests, error metrics, WAF logs, audit events.
Data	KMS encryption, TLS, least-privilege DB roles, backups, tokenization, no real data.	Database logs, backup alerts, restore tests, data-access audit.
Operations	SLOs, runbooks, approved changes, rollback, incident roles, backup/DR.	Alerts, synthetic checks, game days, incident records, postmortems.

9.2 Mandatory Security Gate Policy

Gate	Pull request behavior	Release behavior	Exception rule
Secrets	Any verified or high-confidence secret blocks merge.	No release if secret exposure is unresolved; rotate and purge history as required.	No normal exception. False positives require rule tuning and peer approval.
SAST	Critical/high security findings block; medium findings require triage.	No critical/high unresolved findings in changed code.	Time-bound risk acceptance, owner, compensating control, expiry <= 30 days for training.
Dependencies/SCA	Critical/high fixable vulnerabilities block; license denylist blocks.	Release must meet vulnerability SLA and dependency policy.	Document no-fix condition, exploitability, compensating controls, and upgrade plan.
IaC / cloud config	Critical/high misconfiguration or policy violation blocks.	Apply is blocked until corrected or approved.	Security owner approval and explicit policy waiver with expiry.
Container image	Critical/high fixable vulnerabilities block.	Image must have SBOM, scan, signature, and provenance.	Digest-specific waiver only; never wildcard a repository.
Kubernetes policy	Schema, lint, and mandatory policy violations block.	Admission controller rejects non-compliant workloads.	Namespace/policy exception must be scoped, owned, documented, and temporary.
DAST/API	High findings block promotion; medium findings require triage.	Authenticated scan required in staging before production.	Same risk-acceptance requirements; no suppression without evidence.
Operational verification	Failed probes, smoke tests, error-rate/latency breach block promotion.	Automatic/manual rollback is initiated.	Emergency release uses break-glass change record and retrospective review.

9.3 Vulnerability Management Workflow

1. Finding is created with tool, component, version/digest, severity, exploitability context, owner, and SLA.
2. Team validates whether the finding is reachable and applicable; evidence is attached. Severity may not be lowered solely to pass the pipeline.
3. Fix through dependency update, code change, configuration change, base-image update, or removal of the vulnerable component.
4. If no immediate fix exists, create a risk exception containing business justification, compensating controls, approver, expiry, and review date.
5. Retest the exact artifact or configuration. Close only after evidence shows the finding is absent or accepted.
6. Report open security debt by age, severity, owner, and environment. Expired exceptions automatically fail the release gate.

9.4 Identity and Access Requirements

- All human administrators use SSO and MFA. Shared user accounts are prohibited.

- GitHub Actions authenticates to AWS through OIDC. AWS access keys in GitHub secrets are prohibited.
- Each workflow and workload uses a dedicated role with least-privilege permissions and restrictive trust conditions.
- Production apply and production GitOps approval require separation between author and approver.
- EKS access uses access entries and Kubernetes RBAC groups. cluster-admin is restricted to break-glass roles.
- Argo CD uses SSO/RBAC and AppProjects. Application teams can sync only approved applications/namespaces.
- Database access uses separate application, migration, read-only support, and admin roles. Application roles cannot create users or bypass audit.
- Access reviews are performed at project midpoint and before final submission; evidence lists removals and approvals.

9.5 Secrets and Key Management

- Store secrets in AWS Secrets Manager and encrypt with customer-managed KMS keys where the profile requires it.
- Use External Secrets Operator or Secrets Store CSI Provider with workload identity. Do not store base64-encoded Kubernetes Secrets in Git.
- Rotate database/application secrets and demonstrate that the service consumes the new value without image rebuild.
- Use separate KMS keys or clear key policies for state, data, logs, secrets, and signing where practical.
- Never expose Terraform sensitive outputs, OIDC tokens, kubeconfig, database passwords, or signing material in logs/artifacts.
- Create a secret incident runbook covering detection, revocation, rotation, repository history cleanup, artifact invalidation, and notification.

9.6 GitHub Actions Security Skeleton

Illustrative workflow control pattern (replace placeholders with reviewed, pinned SHAs).

```
name: reusable-secure-build
on:
  workflow_call:
permissions:
  contents: read
  security-events: write # only when SARIF upload is required
concurrency:
  group: build-${{ github.ref }}
  cancel-in-progress: true
jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@<FULL_COMMIT_SHA>
      - run: make lint test security-checks
  publish:
    if: github.ref == 'refs/heads/main' && github.event_name == 'push'
    needs: validate
    permissions:
      contents: read
      id-token: write # short-lived OIDC only in this trusted job
      packages: write
      attestations: write
    environment: release
    steps:
      - uses: actions/checkout@<FULL_COMMIT_SHA>
      - run: make build sbom scan sign attest publish
```

10. Monitoring, Logging, Tracing, and SRE

10.1 Telemetry Architecture

Application services emit OpenTelemetry traces and metrics and structured JSON logs. An OpenTelemetry Collector receives, enriches, samples, and exports telemetry. Prometheus scrapes platform/application metrics; Loki stores logs; Tempo stores traces; Grafana provides cross-signal exploration; Alertmanager routes alerts; CloudWatch captures AWS service/control-plane telemetry. Enterprise teams may forward selected signals and security events to a central SIEM.

10.2 Required Metrics

Category	Examples
Application RED	Request rate, error rate, latency histograms, status code, endpoint, dependency latency, retry count, timeout count.
Resource USE	CPU utilization/saturation, memory utilization/OOM, node filesystem, network, pod restarts, pending pods, throttling.
Finance-domain	Synthetic transaction requests, accepted/rejected/risk-reviewed, ledger completion duration, idempotent replay count, failure reason class.
Database	Connections, CPU, storage, query latency, deadlocks, replication/backup status, connection pool saturation.
Ingress/WAF	ALB target health, 4xx/5xx, response time, rejected requests, WAF rule matches, rate-limit events.
CI/CD	Workflow duration/success, queue time, scan failures, deployment frequency, lead time, rollback count, change failure rate.
Security	Critical/high findings, policy violations, denied admissions, Falco events, GuardDuty/Security Hub findings, secret-rotation age.
SLO	Availability, latency objective compliance, error-budget burn rate, synthetic transaction success.

10.3 Structured Logging Standard

Field	Requirement
timestamp	UTC ISO 8601 with millisecond precision.
level	Standardized severity: DEBUG, INFO, WARN, ERROR. Production default excludes DEBUG.
service/environment/version	Service name, deployment environment, semantic version, and image digest/short digest.
trace_id/span_id/correlation_id	Enable cross-signal investigation and request tracing.
transaction_token	Synthetic/tokenized identifier only; never PAN, account number, name, or other real sensitive data.
event/action/outcome	Stable event taxonomy for audit and alerting.
actor/client	Non-sensitive service/client identity or pseudonymous test user.
error	Error class and safe message; stack trace only where access/retention are controlled.
prohibited data	Secrets, authorization headers, session tokens, private keys, raw request bodies containing sensitive-like fields.

10.4 SLO and Alert Design

SLO/alert	Training target / rule	Response
Availability SLO	99.9% monthly for valid transaction API requests.	Track error budget; page only on fast/slow burn or sustained outage.
Latency SLO	95% of valid API requests below 500 ms; team may define stricter p99 target.	Investigate application, ingress, database, and saturation signals.
Ledger completion	99% of accepted synthetic transactions reach final status within 5 seconds under normal test load.	Alert on backlog and processing duration.
Error-rate alert	Critical when valid-request error rate exceeds 5% for 5 minutes; warning at lower sustained threshold.	Run API error triage and consider rollback.
Pod crash loop	Repeated restart or unavailable replicas beyond stabilization period.	Check events, logs, config, secret, resource, and dependency health.
Database saturation	Connection pool > 85%, low storage, or high query latency.	Throttle, scale, tune, or fail safely; protect data integrity.
Security finding	Critical GuardDuty/Falco/admission/signature event.	Follow security incident runbook and preserve evidence.

SLO/alert	Training target / rule	Response
Backup failure	Missed or failed backup/restore test.	Escalate; do not treat backup configuration as proof until restore is tested.

10.5 Required Dashboards

- Executive/service health: SLO status, error budget, releases, incidents, and top risks.
- Application RED: requests, errors, latency, dependency calls, transaction outcomes, and idempotent retries.
- Kubernetes platform: nodes, workloads, requests/limits, restarts, pending pods, autoscaling, and admission denials.
- Database: connections, latency, CPU/storage, backups, and slow queries.
- Ingress/WAF: traffic, response codes, target health, WAF blocks, and rate limiting.
- DevSecOps: PR cycle time, workflow success/duration, deployment frequency, lead time, scan findings, and rollback rate.
- Security: findings by source/severity/status, policy violations, runtime events, and exception expiry.
- Trace explorer: sample transaction trace with links to logs and relevant metrics.

11. Resilience, Backup, Disaster Recovery, and Rollback

11.1 Resilience Requirements

- Run at least two replicas for stateless services and spread replicas across nodes/AZs when capacity permits.
- Use readiness, liveness, and startup probes with realistic thresholds; avoid probes that create harmful load.
- Use PodDisruptionBudgets and graceful termination. Test node drain and rolling upgrade behavior.
- Use HPA and define maximum scaling to protect downstream dependencies. Consider queue/backpressure for the ledger worker.
- Use bounded retries with jitter and idempotency. Never allow retries to duplicate ledger events.
- Protect RDS with backups, encryption, maintenance windows, monitoring, and tested restore procedures.
- Keep infrastructure and desired state reconstructable from Git and Terraform; identify any external state that needs backup.
- Use release health criteria and rollback automation. Maintain the previous approved digest and configuration.

11.2 Required Failure Exercises

Exercise	Expected behavior	Evidence
Delete an application pod	Replica set replaces it; service remains available or recovers within SLO.	kubectl events, metrics, synthetic check.
Drain a worker node	PDB/topology rules maintain service; pods reschedule safely.	Drain transcript, availability dashboard.
Deploy broken readiness	Argo/Kubernetes marks release unhealthy; promotion stops and rollback occurs.	Argo history, workflow, incident timeline.
Database unavailable	Application fails safely, emits useful telemetry, avoids duplicate writes, and recovers after restoration.	Logs/traces/metrics and transaction integrity check.
Expired/rotated secret	Rotation procedure updates workload without repository change or image rebuild.	Secrets Manager version and rollout evidence.
High latency/load	Autoscaling and alerting respond without uncontrolled retry storm.	k6 report, HPA events, dashboards.
Restore database	Restore to isolated environment and validate record count/integrity.	Timed restore report and checksum/query validation.

11.3 Rollback Rules

Rollback Principle

Application rollback is a GitOps desired-state change, not a manual replacement of running resources. Database schema changes must follow expand-and-contract or forward-compatible migrations so that the previous application version remains operable during the release window.

12. Financial-Industry Standards and Control Mapping

The project uses standards as control-design references and evidence categories. Teams must map only controls they actually implemented and must state gaps. The following references were current when this plan was prepared; organizations must validate applicability and current versions with legal, compliance, risk, and security functions.

12.1 Standards Mapping

Reference	Primary themes	Project evidence examples
PCI DSS v4.0.1	Network security, secure configuration, data protection, encryption, secure development, access control, authentication, logging, testing, policies.	WAF/NetworkPolicy, KMS/TLS, ASVS requirements, CI gates, least privilege, audit logs, DAST, vulnerability process, evidence pack.
NIST CSF 2.0	Govern, Identify, Protect, Detect, Respond, Recover.	Governance/ADRs/risk register; inventory; preventive controls; monitoring; incident runbooks; backup/restore and lessons learned.
NIST SSDF 1.1	Prepare the organization, protect software, produce well-secured software, respond to vulnerabilities.	Repository governance, build isolation, secure coding/tests, SBOM/provenance, vulnerability workflow, exception and remediation evidence.
NIST SP 800-53 Rev. 5 Release 5.2.0	Access control, audit/accountability, configuration, incident response, risk assessment, system acquisition, system integrity, supply chain.	IAM/RBAC, CloudTrail/logging, IaC/Config, incident exercises, risk register, secure SDLC, scanning/signing/provenance.
FFIEC DA&M 2024	Governance, project management, SDLC, development/acquisition/maintenance, change management, supply-chain risk, resilience.	Team charter, approved tool choices, controlled lifecycle, change records, dependency governance, testing, maintenance and DR procedures.
EU DORA	ICT risk management, incident management/reporting, resilience testing, third-party ICT risk, information sharing.	Risk/control model, incident runbooks/evidence, game days/restore tests, vendor/tool inventory and due diligence assumptions.
ISO/IEC 27001:2022	ISMS risk-based controls, access, cryptography, operations, secure development, supplier relationships, incident and continuity.	Risk register, access review, KMS, operational procedures, DevSecOps gates, third-party inventory, incident and DR runbooks.
NYDFS 23 NYCRR 500 (where applicable)	Cybersecurity program, access, application security, vulnerability testing, audit trails, incident and third-party controls.	Secure SDLC, access review, continuous scanning, audit/event records, incident workflow, vendor/tool risk records.
OWASP ASVS 5.0 / Top 10:2025	Application security verification and awareness risks.	Application requirements, SAST/DAST/API tests, secure configuration, logging, auth, input validation, supply-chain controls.
CIS Benchmarks	Consensus secure configuration for AWS, Kubernetes, operating systems, databases, and containers.	Checkov/Config rules, EKS/AWS hardening checklist, kube-bench where applicable, hardened node/container configuration.
Kubernetes Pod Security Standards	Privileged, Baseline, and Restricted security profiles.	Pod Security Admission labels and Kyverno policies aligned to Restricted with documented exceptions.
SLSA 1.2	Incremental software supply-chain integrity and provenance.	Hosted trusted build, provenance/attestation, artifact verification, immutable digest, protected source/build definitions.

12.2 Minimum Evidence-to-Control Examples

Control objective	Evidence
Authorized change	Pull request, reviewers, CODEOWNERS result, protected environment approval, Git commit/tag.
Infrastructure change control	Terraform plan, scan results, apply approval, workflow run, state/version record.
Secure software release	Tests, SAST/SCA, image scan, SBOM, signature, provenance, release notes.
Deployment integrity	GitOps PR, approved digest, Argo sync/health history, signature verification/admission result.
Least privilege	AWS IAM policies/trust, GitHub permissions, Kubernetes RBAC, Argo AppProject, database grants.
Audit trail	CloudTrail, GitHub audit/deployment history, Argo history, Kubernetes audit logs, application audit events.
Detection and response	Alert rules, test alerts, incident record, timeline, containment/recovery actions, postmortem.
Operational resilience	Backup configuration, restore report, failure exercise, RTO/RPO measurement, corrective actions.
Third-party risk	Tool/dependency inventory, pinned versions, SBOM, license results, vendor/service assumptions, update process.
Risk acceptance	Finding, impact, compensating control, approver, owner, expiry, retest/closure evidence.

13. Testing and Validation Strategy

Test layer	Tools / approach	When	Blocking criteria
Code formatting/lint	Language linters, Checkstyle/Spotless/ruff/eslint, Hadolint.	Local + every PR.	Any error.
Unit tests	JUnit/pytest/Jest and coverage.	Every PR.	Failed tests; critical path below team coverage threshold.
Integration tests	Testcontainers or ephemeral dependencies.	Every trusted PR/main build.	Failed integration or data-integrity assertion.
Contract/API	OpenAPI validation, Schemathesis optional, consumer contract tests.	PR and staging.	Breaking or undocumented contract.
Terraform	fmt, validate, TFLint, Checkov, plan, optional Terratest.	PR; plan before apply.	Validation/policy failure or unreviewed destructive change.
Ansible	ansible-lint, syntax check, check mode, Molecule.	PR where used.	Lint/syntax/idempotency failure.
Helm/Kubernetes	helm lint/template, values schema, kubeconform, kube-linter, Conftest.	PR and GitOps PR.	Schema or mandatory policy failure.
Secrets/SAST/SCA	Gitleaks, Semgrep/CodeQL, Dependabot/Trivy fs, license policy.	Every PR and scheduled scan.	Per security gate policy.
Container	Image scan, non-root/read-only checks, SBOM, signature/provenance verification.	Release build and pre-deploy.	Per security gate; missing trust metadata.
DAST	OWASP ZAP baseline on dev; authenticated/full in staging.	After deployment.	High finding or scan failure.
Smoke/E2E	Health, synthetic transaction, ledger completion, UI/API flow.	Every deployment.	Any required flow failure.
Performance	k6 load, stress, and soak profile.	Staging before final promotion.	SLO/threshold breach without approved capacity decision.
Resilience	Pod/node/dependency/release failure tests and restore.	Scheduled game day.	Data integrity failure or unrecoverable service.
Security tabletop	Credential, image, secret, runtime, and third-party scenarios.	Before final handover.	No owner/runbook/evidence for critical scenario.

13.1 Performance Test Profile

- Warm-up: 2 minutes at low rate to initialize connections and caches.
- Baseline: agreed normal load for 10 minutes; validate latency, errors, and ledger completion.
- Spike: 3x baseline for 2 minutes; validate autoscaling and safe degradation.
- Soak: at least 30 minutes for the final project; monitor leaks, connection pools, queue growth, and cost.
- Record test commit, image digest, environment, dataset, k6 script, thresholds, and dashboard snapshots.

14. Documentation, Evidence, and Runbooks

14.1 Mandatory Documentation Set

Artifact	Minimum content
README.md	Purpose, prerequisites, architecture summary, quick start, repositories, environments, pipeline, support, and teardown.
docs/architecture/*	Context, container/component diagrams, data flow, trust boundaries, AWS topology, network flows.
docs/adr/*	Tool choices, repo model, secrets, GitOps, observability, policy engine, release strategy, exceptions.
docs/security/*	Threat model, ASVS requirements, IAM matrix, vulnerability policy, supply-chain design, exception process.
docs/sre/*	SLIs/SLOs, error budgets, dashboards, alert design, capacity, operational readiness.
docs/runbooks/*	Deployment rollback, Terraform state recovery, Argo recovery, secret rotation, incident, alerts, DB restore, DR.
docs/compliance/*	Control mapping, evidence index, gaps, assumptions, access review, third-party inventory.
docs/testing/*	Test strategy, test data, performance profile, DAST, resilience game day.
docs/handover.md	Ownership, dependencies, upgrade cadence, routine operations, known risks, backlog, cost controls.
evidence/*	Release-specific immutable or integrity-protected evidence bundles.

14.2 Evidence Pack Structure

Recommended evidence directory.

```
evidence/
  final-release-v1.0.0/
    00-index.md
    01-change-request/
    02-source-and-reviews/
    03-terraform-plan-and-apply/
    04-ci-test-results/
    05-security-scans/
    06-sbom-signature-provenance/
    07-gitops-promotion/
    08-argocd-deployment/
    09-post-deploy-verification/
    10-observability-snapshots/
    11-access-review/
    12-backup-restore-game-day/
    13-risk-exceptions/
    14-incident-and-postmortem/
```

14.3 Mandatory Runbooks

Runbook	Required decision points
Failed deployment / rollback	Trigger, blast radius, pause promotion, select previous digest, validate health, communicate, preserve evidence, close/postmortem.
Terraform state recovery	Detect corruption/lock, stop applies, identify last good version, recover safely, validate plan, approve, document.
Argo CD recovery	Loss of service/credentials/config, bootstrap path, restore declarative config, validate apps, rotate secrets if required.
Secret exposure / rotation	Contain, revoke, rotate, invalidate artifacts, purge or rewrite history if needed, scan, communicate, review.
Security incident	Severity, roles, evidence preservation, containment, eradication, recovery, regulatory escalation decision, postmortem.
Database restore	Declare restore, select recovery point, isolate, restore, validate integrity, switch/test, document RTO/RPO.
Certificate expiry	Detection, renewal path, emergency fallback, validation, root cause, preventive alert.
High error rate / latency	Confirm alert, correlate release, inspect RED/USE and traces, mitigate/rollback/scale, validate SLO.

Runbook	Required decision points
Cost anomaly / teardown	Identify resource, owner, safe stop/destroy sequence, preserve evidence/state, verify no orphaned resources.

15. Delivery Schedule, Roles, and Governance

15.1 Eight-Week Delivery Plan

Week	Primary milestone	Exit criteria
1	Governance, repositories, application baseline, threat model	Repo controls configured; architecture/ADR/threat model approved; local app tests pass.
2	Terraform bootstrap, OIDC, network and shared services	Encrypted remote state, plan/apply workflow, OIDC roles, VPC baseline.
3	EKS, ECR, RDS, platform add-ons, policy baseline	Cluster/data provisioned; Argo/add-ons/policies operational; drift and denial tests pass.
4	CI gates and trusted artifact build	PR gates, tests/scans, SBOM, signature, provenance, immutable ECR digest.
5	GitOps CD and environment promotion	Dev/staging deployment, smoke/DAST, promotion PR, rollback demonstrated.
6	Observability, SLOs, alerts, runtime security	Dashboards, alerts/runbooks, traces/logs/metrics correlation, security findings routed.
7	Performance, resilience, restore, compliance evidence	k6 results, failure exercises, DB restore, control matrix, access review.
8	Hardening, peer audit, final release and demonstration	Clean deployment, final evidence pack, no blocking findings, demo and submission.

15.2 Recommended Team Roles

Role	Responsibilities
Project / Delivery Lead	Scope, schedule, risks, backlog, change records, dependency management, final coordination.
Platform / IaC Engineer	Terraform modules/stacks, AWS networking/IAM/EKS/RDS, state, drift, cost and teardown.
CI/CD / Release Engineer	GitHub Actions, artifact build, GitOps promotion, Argo CD, release evidence, rollback.
Security Engineer	Threat model, gates, policies, findings, supply-chain controls, IAM review, incident/tabletop.
SRE / Observability Engineer	OpenTelemetry, dashboards, SLOs, alerts, performance, resilience, runbooks.
Application Engineer	API/services, tests, Dockerfiles, migrations, telemetry, idempotency, secure coding.
Audit / Documentation Owner	ADRs, control matrix, evidence index, access review, peer audit, handover.

15.3 RACI for Critical Activities

Activity	App	Platform	Security	SRE	Release	Lead/Audit
Application change	R	C	C	C	A	I
Terraform apply	I	R	C	C	A	I
Security gate policy	C	C	A/R	C	C	I
Production promotion	I	C	C	C	R	A
Incident response	C	C	A/R	R	C	I
Backup/restore	I	R	C	A/R	I	I
Evidence/control mapping	C	C	C	C	C	A/R

16. Grading Rubric and Definition of Done

16.1 Grading Rubric (100 Points)

Category	Points	Excellent evidence
Architecture and IaC	20	Modular secure Terraform; repeatable multi-environment design; remote state/OIDC; hardened AWS/EKS/RDS; clean plans; drift/teardown.
CI/CD and GitOps	20	Protected/reusable workflows; build-once promote-many; immutable digest; Argo drift/health; approvals; safe promotion/rollback.
Security and supply chain	25	Threat model; strong gates; least privilege; secrets; SBOM/sign/provenance; admission/runtime/cloud security; managed exceptions.
Observability and SRE	15	OTel; meaningful dashboards/alerts; SLO/error budget; cross-signal investigation; performance and operational readiness.
Resilience and recovery	8	Multi-AZ-aware workload design; failure tests; rollback; timed backup restore; documented RTO/RPO.
Documentation and evidence	8	README-driven setup; ADRs/runbooks; control mapping; evidence index; independent audit traceability.
Final demonstration and teamwork	4	Clear roles, reproducible live flow, honest gap/risk discussion, lessons learned.

16.2 Automatic Fail / Major Deduction Conditions

- Real financial/customer data, production credentials, cardholder data, or secrets are committed, logged, captured, or submitted.
- Infrastructure or production-simulated application is created/deployed primarily through manual console or direct kubectl without recorded exception.
- Cloud access keys are stored in GitHub instead of OIDC, or broad administrator permissions are used without a documented temporary reason.
- No security gates, no artifact scan/SBOM/signature, or critical/high findings are ignored without an approved exception.
- Production artifact is rebuilt instead of promoting the tested digest.
- No rollback or restore demonstration.
- Documentation cannot reproduce the environment or evidence cannot trace a release.
- Fabricated scan, test, monitoring, or compliance evidence.

16.3 Definition of Done

- All infrastructure and platform configuration is version-controlled and reproducible.
- Protected repositories and environments enforce review and status checks.
- CI runs tests and all mandatory security scans; trusted release produces immutable signed artifact, SBOM, and provenance.
- GitOps deploys the same digest through dev, staging, and production simulation; direct deployment is not normal practice.
- Application, cluster, cloud, and data controls meet the documented baseline or have approved gaps.
- Dashboards, SLOs, alerts, runbooks, and synthetic checks are operational.
- Rollback, node/pod failure, and database restore are tested and evidenced.
- Final release has no unresolved blocking findings; all exceptions are valid and unexpired.
- Documentation and evidence support an independent walkthrough from source change to production verification.
- Teardown/cost-control procedure is tested and leaves no unintended resources.

17. Final Demonstration and Submission Checklist

17.1 Recommended 30-45 Minute Demonstration

1. Business scenario, data restrictions, architecture, trust boundaries, and selected deployment profile.
2. GitHub organization controls, repository separation, CODEOWNERS, environment approvals, and OIDC trust.
3. Open a pull request with a safe change; show tests and security gates. Demonstrate one deliberate blocked issue.
4. Merge the approved change; show trusted build, SBOM, image scan, signature, provenance, and immutable ECR digest.
5. Show the GitOps promotion PR and Argo CD reconciliation to dev/staging. Run smoke and synthetic transaction tests.
6. Promote the same digest to production simulation with approval. Show deployment strategy and post-deploy verification.
7. Trace one transaction across metrics, logs, and traces. Show SLO/error budget and business dashboard.
8. Trigger a controlled failure; show alert, runbook use, rollback/recovery, and incident evidence.
9. Show database restore evidence, access review, control mapping, risk register, exceptions, and final evidence index.
10. Close with cost controls, known gaps, lessons learned, and production improvements.

17.2 Submission Checklist

- ☐ GitHub organization/repository links with reviewer access.
- ☐ Final semantic release tag and immutable image digest.
- ☐ Architecture diagrams, threat model, ADRs, and data-handling statement.
- ☐ Terraform modules/stacks, state design, workflows, and clean plan evidence.
- ☐ CI workflows, security gate configuration, scan results, SBOM, signature, provenance.
- ☐ GitOps repository, Argo definitions, Helm chart, environment values, and promotion history.
- ☐ Observability configuration, dashboards, alerts, SLOs, and runbooks.
- ☐ Performance report, resilience game-day report, rollback and restore evidence.
- ☐ Control matrix, evidence index, access review, risk register, exceptions, incident/postmortem.
- ☐ Final report, demo script/recording if required, teardown instructions, and self-assessment rubric.

18. Risks, Constraints, and Stretch Goals

18.1 Common Project Risks and Mitigations

Risk	Impact	Mitigation
Too many tools	Integration complexity and unfinished core controls.	Implement mandatory path first; add enterprise/stretch tools only after Definition of Done.
Cloud cost	Unexpected EKS/NAT/RDS/logging charges.	Budgets/alerts, small instances, scheduled teardown, lifecycle/retention, VPC endpoint cost review, destroy workflow.
Runner compromise	Credentials or supply-chain compromise.	Separate trust boundaries, ephemeral runners, no secrets for untrusted PRs, minimal permissions, OIDC, action pinning.
False-positive fatigue	Teams bypass security gates.	Tune rules, classify findings, centralize exceptions, measure actionable findings, never suppress globally without review.
Over-broad IAM	Large blast radius.	Role-per-workflow/workload, scoped trust, policy review, Access Analyzer, access review.
GitOps secret leakage	Secrets enter repository/history.	External Secrets/CSI, Gitleaks, push protection, encrypted secret solution only if explicitly approved.
Database migration failure	Rollback blocked or data corruption.	Expand-contract migrations, backups, pre-checks, migration job, forward-fix plan, restore test.
Observability cost/noise	High ingestion and alert fatigue.	Sampling, structured fields, retention, cardinality controls, actionable alerts, dashboards-as-code.
Compliance overclaim	Misleading project conclusion.	Use "aligned/mapped," identify gaps, and state that certification requires formal scope and independent assessment.
Manual undocumented steps	Non-reproducible environment.	Automate or record as bootstrap with owner, reason, and plan to eliminate.

18.2 Stretch Goals

- Argo Rollouts canary analysis with Prometheus metrics and automated rollback.
- Private EKS endpoint with self-hosted ephemeral runners managed by Actions Runner Controller in a dedicated runner account/VPC.
- AWS Organizations multi-account landing zone and centralized security/log archive accounts.
- HashiCorp Vault with dynamic database credentials and workload identity.
- Cilium/Hubble network observability and L7 policy, or service mesh mTLS with documented operational trade-offs.
- Chaos Mesh/Litmus automated resilience experiments and scheduled game days.
- OpenCost/Infracost dashboards and policy-based cost gates.
- SIEM integration with Splunk/Elastic/Sentinel and correlation rules for CI/CD, cloud, cluster, WAF, and application audit events.
- Automated compliance evidence collection and policy reports published per release.
- Multi-region pilot, Route 53 failover, cross-region backup restore, and measured regional RTO/RPO.
- FIPS endpoints/validated cryptography requirements where organizational and regional needs justify them.
- Progress toward SLSA Build Level 2/3 with hardened hosted builds and enforced provenance verification.

Appendix A - Repository Structures

A.1 Application Repository

```
financeguard-app/  
  .github/  
    workflows/  
      ci.yml  
      release.yml  
      reusable-ci.yml  
  CODEOWNERS  
  dependabot.yml  
  services/  
    transaction-api/  
    risk-service/  
    ledger-worker/  
  openapi/financeguard.yaml  
  charts/financeguard/  
  tests/  
    integration/  
    contract/  
    performance/  
    security/  
  scripts/  
  docs/  
  Makefile  
  SECURITY.md  
  CONTRIBUTING.md  
  README.md
```

A.2 Platform Repository

```
financeguard-platform/  
  .github/workflows/  
    terraform-plan.yml  
    terraform-apply.yml  
    terraform-drift.yml  
    platform-validate.yml  
  bootstrap/  
  terraform/  
    modules/  
      network/  
      iam-oidc/  
      eks/  
      ecr/  
      rds/  
      security-baseline/  
      observability-aws/  
  environments/  
    dev/  
    staging/  
    prod/  
  ansible/  
    roles/  
    playbooks/  
  platform/  
    addons/  
    policies/kyverno/  
    rbac/  
    network-policies/  
  observability/  
  docs/  
  README.md
```

A.3 GitOps Repository

```
financeguard-gitops/  
  argocd/  
    projects/  
      applicationsets/  
environments/  
  dev/financeguard/  
    values.yaml  
    image-digest.yaml  
  staging/financeguard/  
    values.yaml  
    image-digest.yaml  
  prod/financeguard/  
    values.yaml  
    image-digest.yaml  
policies/  
docs/promotion.md  
.github/CODEOWNERS  
.github/workflows/validate-gitops.yml  
README.md
```

Appendix B - GitHub Settings Checklist

- ☐ Default branch protected; PR required; force push/deletion disabled.
- ☐ Required checks: CI, security, IaC/manifest policy, GitOps validation.
- ☐ CODEOWNERS review required for workflows, Terraform, policies, production GitOps paths, migrations.
- ☐ Production environment reviewers configured; self-review disabled; deployment branch/tag restrictions enabled.
- ☐ Workflow default permissions set to read-only; organization action policy/allowlist configured.
- ☐ Third-party actions pinned to full commit SHAs; Dependabot/Renovate monitors action updates.
- ☐ Secret scanning and push protection enabled where available; Gitleaks workflow mandatory.
- ☐ Dependabot alerts and dependency graph enabled; private registry access scoped.
- ☐ Repository audit logs/release history retained according to organization policy.
- ☐ Issue templates and labels for change, incident, vulnerability, risk exception, and audit evidence.

Appendix C - Terraform Backend and Workflow Pattern

Illustrative S3 backend pattern. Bootstrap the bucket/key before using the backend; do not hard-code credentials.

```
terraform {  
  backend "s3" {  
    bucket    = "<bootstrap-created-state-bucket>"  
    key       = "financeguard/<environment>/terraform.tfstate"  
    region    = "<approved-region>"  
    encrypt   = true  
    kms_key_id = "<state-kms-key-arn>"  
    use_lockfile = true  
  }  
}
```

The state bucket must have public-access block, versioning, encryption, access logging, least-privilege bucket/KMS policies, and recovery documentation. DynamoDB-based locking should not be selected for new projects when the approved Terraform version supports native S3 lockfiles.

Appendix D - Kubernetes Security Values Pattern

Illustrative Helm values; adapt to the application and validate with policy and performance tests.

```
replicaCount: 2
image:
  repository: <account>.dkr.ecr.<region>.amazonaws.com/financeguard-api
  digest: "sha256:<immutable-digest>"
podSecurityContext:
  runAsNonRoot: true
  seccompProfile:
    type: RuntimeDefault
securityContext:
  allowPrivilegeEscalation: false
  readOnlyRootFilesystem: true
  capabilities:
    drop: ["ALL"]
resources:
  requests: {cpu: "100m", memory: "128Mi"}
  limits: {cpu: "500m", memory: "512Mi"}
probes:
  readiness: {path: /ready, initialDelaySeconds: 5}
  liveness: {path: /health, initialDelaySeconds: 20}
networkPolicy:
  enabled: true
```

Appendix E - Local and CI Command Checklist

- Application: make fmt lint test integration-test
- Secrets: gitleaks detect --source . --redact
- SAST: semgrep ci --config auto (or approved ruleset)
- SCA/filesystem: trivy fs --scanners vuln,misconfig,secret,license .
- Terraform: terraform fmt -check -recursive; terraform validate; tflint --recursive; checkov -d terraform/
- Ansible: ansible-lint; ansible-playbook --syntax-check; ansible-playbook --check (where safe)
- Helm: helm lint charts/financeguard; helm template ... | kubeconform -strict; kube-linter lint ...
- Policy: conftest test <rendered-manifests-or-plan-json>
- Image: trivy image --severity HIGH,CRITICAL --ignore-unfixed <image@digest>
- SBOM: syft <image@digest> -o cyclonedx-json > sbom.cdx.json
- Signing: cosign sign <image@digest>; cosign verify <identity-policy> <image@digest>
- DAST: zap-baseline.py -t <dev-url>; authenticated/full scan in staging
- Performance: k6 run tests/performance/transaction.js
- Kubernetes verification: kubectl rollout status; kubectl wait; smoke tests; Argo health/sync check

Appendix F - Evidence Index Template

Evidence ID	Control / decision	Source	Owner	Retention / integrity	Status
EV-001	Approved application change	GitHub PR and reviews			
EV-002	Infrastructure plan/apply	GitHub Actions + Terraform			
EV-003	Security scan summary	CI SARIF/reports			
EV-004	SBOM/signature/provenance	ECR/GitHub/evidence bundle			
EV-005	GitOps promotion and approval	GitHub PR + environment			
EV-006	Argo deployment and health	Argo CD history			
EV-007	Post-deployment verification	Smoke/DAST/metrics			
EV-008	Access review	GitHub/AWS/K8s/Argo/DB			
EV-009	Backup restore	Restore report			
EV-010	Incident/rollback exercise	Incident record/postmortem			

Appendix G - Authoritative Reference Sources

R1. PCI Security Standards Council - PCI DSS v4.0.1 Document Library:

https://www.pcisecuritystandards.org/document_library/

R2. NIST Cybersecurity Framework 2.0: <https://www.nist.gov/publications/nist-cybersecurity-framework-csf-20>

R3. NIST SP 800-218 - Secure Software Development Framework (SSDF) Version 1.1: <https://csrc.nist.gov/pubs/sp/800/218/final>

R4. NIST SP 800-53 Rev. 5 / Release 5.2.0: <https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final>

R5. FFIEC - Development, Acquisition, and Maintenance booklet announcement (2024):

<https://www.ffiec.gov/news/press-releases/2024/pr-09-29>

R6. EUR-Lex - Digital Operational Resilience Act (EU) 2022/2554: <https://eur-lex.europa.eu/eli/reg/2022/2554/oj/eng>

R7. ISO/IEC 27001:2022 - Information security management systems: <https://www.iso.org/standard/27001>

R8. New York DFS - Cybersecurity Regulation resource center: https://www.dfs.ny.gov/industry_guidance/cybersecurity

R9. OWASP Application Security Verification Standard 5.0:

<https://owasp.org/www-project-application-security-verification-standard/>

R10. OWASP Top 10:2025: <https://owasp.org/Top10/2025/>

R11. Kubernetes Pod Security Standards: <https://kubernetes.io/docs/concepts/security/pod-security-standards/>

R12. CIS Benchmarks - AWS and Kubernetes: <https://www.cisecurity.org/cis-benchmarks>

R13. SLSA Specification v1.2: <https://slsa.dev/spec/v1.2/>

R14. GitHub Actions - OIDC with AWS and secure use:

<https://docs.github.com/actions/security-for-github-actions/security-hardening-your-deployments/configuring-openid-connect-in-amazon-web-services>

R15. GitHub Actions secure use reference: <https://docs.github.com/en/actions/reference/security/secure-use>

R16. Argo CD documentation and GitOps model: <https://argo-cd.readthedocs.io/en/stable/>

R17. Terraform S3 backend and state locking: <https://developer.hashicorp.com/terraform/language/backend/s3>

R18. Amazon EKS best practices - identity and access management:

<https://docs.aws.amazon.com/eks/latest/best-practices/identity-and-access-management.html>

R19. Sigstore Cosign container signing: https://docs.sigstore.dev/cosign/signing/signing_with_containers/

R20. OpenTelemetry documentation: <https://opentelemetry.io/docs/>

Version Management Note

The project must pin approved tool and provider versions in code and upgrade through pull requests. "Latest" must never mean unreviewed floating versions in CI/CD or production. Revalidate references and service behavior before each new training batch.